



AAK ACADEMY MANAGEMENT SYSTEM (AAMS)

Database Design Document

V 2.0

By

Sr.	Group Member Name	Roll No.
1	SALMAN SAFDAR	NUM-BSCS-2024-70
2	MUHAMMAD HARIS	NUM-BSCS-2024-49
3	FARWA IMRAN	NUM-BSCS-2024-22

Department of Computer Sciences

Namal University, Mianwali, Pakistan

Submission Date: 21st June, 2026

Course	Database Systems
Database Name	aak_academy
DBMS	MySQL 8.x
Frontend	HTML5, CSS3, JavaScript
Backend	Node.js, Express.js
Database Driver	mysql2 (connection pool)
GitHub	github.com/bscs24f22-eng/DataBase-Project

REVISION HISTORY

Date	Version	Description	Approved By
21/06/26	V 2.0	Final submission with full implementation, GUI, and database connectivity	
15/05/26	V 1.0	Initial ERD, conceptual design, relational schema draft	

Contents

1	PROJECT OVERVIEW	4
1.1	INTRODUCTION	4
1.2	PROBLEM STATEMENT	4
1.3	PROJECT OBJECTIVES	4
2	CONCEPTUAL DATABASE DESIGN	6
2.1	ENTITY	6
2.2	DATA DICTIONARY	7
2.2.1	USER	7
2.2.2	STUDENT	7
2.2.3	SUBJECT	7
2.3	BUSINESS RULES	8
2.4	ENTITY RELATIONSHIP DIAGRAM	8
2.4.1	ERD Design Decisions	9
3	LOGICAL DATABASE DESIGN	11
3.1	RELATIONAL SCHEMA	11
3.1.1	Module 1: User Management	11
3.1.2	Module 2: Academic Structure	11
3.1.3	Module 3: Finance	11
3.1.4	Module 4: Learning & Assessment	12
3.1.5	Module 5: Communication & Security	12
3.1.6	Relationship Cardinality Summary	12
3.2	FUNCTIONAL DEPENDENCIES	13
3.3	NORMALIZATION	13
3.3.1	First Normal Form (1NF)	14
3.3.2	Second Normal Form (2NF)	14
3.3.3	Third Normal Form (3NF)	14
3.3.4	Boyce-Codd Normal Form (BCNF)	15
3.3.5	Normalization Summary	15
4	IMPLEMENTATION	16
4.1	LANGUAGE/Framework	16
4.1.1	DBMS Configuration	16
4.1.2	Constraints Implemented	16
4.1.3	Performance Indexes	17
4.2	DATABASE CONNECTIVITY	17
4.2.1	Three-Tier Architecture Flow	17
4.2.2	REST API Endpoints	17

4.2.3	Security Measures	18
4.3	STORED PROCEDURES AND FUNCTIONS	18
4.3.1	Stored Procedure: sp_CreateStudent	19
4.3.2	Trigger: trg_AutoGenerateFeeRecord	20
4.3.3	Key SQL Queries	20
4.4	INTERFACES	21
4.4.1	GUI Design Philosophy	21
4.4.2	Colour Palette	21
4.4.3	Role-Based Navigation	21
4.4.4	Sample Data Statistics	22
4.4.5	Screenshots	22
Testing & Validation		36
Conclusion		37
References		38
Appendix: AI Usage and Prompts		39

CHAPTER 1: PROJECT OVERVIEW

1.1. INTRODUCTION

The **AAK Academy Management System (AAMS)** is a web-based academic management application developed for AAK Academy at Namal University. The system centralizes academic, financial, and communication operations for three user roles: **Admin**, **Teacher**, and **Student**.

Previously, the application used static in-memory JavaScript data. This project implements a **full three-tier architecture** with a MySQL relational database, RESTful API backend, and role-based web GUI. All CRUD operations persist to the database in real time, enabling reliable and consistent data management across the entire academy.

The system is built to support the real-world scenario of an educational institution that manages large volumes of interconnected data including student records, course enrollments, assessments, attendance, and financial transactions.

1.2. PROBLEM STATEMENT

Educational institutions manage large volumes of interconnected data — students, courses, enrollments, fees, assignments, quizzes, attendance, and announcements. Manual or disconnected systems lead to several critical problems:

- **Data Inconsistency:** The same information stored in multiple places leads to conflicting records.
- **Data Duplication:** Redundant entries waste storage and create maintenance challenges.
- **Poor Traceability:** Without audit logs, it is difficult to track who changed what and when.
- **Lack of Role Separation:** Static systems do not enforce access control based on user roles.
- **Manual Financial Tracking:** Fee collection, payment records, and discount management are error-prone when done manually.
- **No Real-Time Updates:** In-memory data is lost on server restart and cannot persist across sessions.

The AAMS system addresses all of these challenges through a normalized relational database with proper constraints, referential integrity, and a role-based access control model.

1.3. PROJECT OBJECTIVES

General Objective: To develop a fully functional, database-driven academic management system that centralizes all administrative, academic, and financial operations of AAK Academy into a secure, consistent, and scalable platform.

Specific Objectives:

- Design a normalized relational database schema satisfying 3NF/BCNF. **[Completed]**
- Convert the ERD entities into MySQL tables with proper constraints. **[Completed]**
- Populate the database with realistic sample data using DML scripts. **[Completed]**
- Build a role-based GUI for Admin, Teacher, and Student users. **[Completed]**
- Connect the GUI to MySQL via a RESTful API using Node.js. **[Completed]**
- Support real-time CRUD operations that persist immediately to the database. **[Completed]**
- Maintain referential integrity, cascade deletes, and audit logging. **[Completed]**

GitHub Repository: <https://github.com/bscs24f22-eng/DataBase-Project>

CHAPTER 2: CONCEPTUAL DATABASE DESIGN

2.1. ENTITY

The following entities are identified as the core components of the AAMS database. Associative/junction entities are not listed here as they emerge from many-to-many relationships.

Sr.	Entity Name	Description
01	USER	An individual registered in the system with a login identity and an assigned role (Admin, Teacher, or Student).
02	ADMIN	A privileged user with full system access who manages all academic and financial records.
03	TEACHER	An academic user responsible for managing subjects, assessments, attendance, and materials.
04	STUDENT	An enrolled learner who participates in subjects, submits assignments, and views academic records.
05	PROGRAM	An academic degree or diploma programme offered by the institution.
06	SUBJECT	A course taught within a programme, assigned to a specific teacher.
07	ASSIGNMENT	A task created by a teacher for students in a subject, with a deadline and marks.
08	QUIZ	A timed online assessment created by a teacher for a specific subject.
09	QUIZ_QUESTION	An individual question belonging to a quiz.
10	LIVE_CLASS	A scheduled online session with a Zoom link, date, and time managed by a teacher.
11	LEARNING_MATERIAL	A file or document uploaded by a teacher for students of a subject.
12	ANNOUNCEMENT	A notice posted by a teacher or admin relevant to a subject or the whole academy.
13	FEE_RECORD	A financial record tracking total fees, discounts, payments, and balance for a student.
14	PAYMENT	An individual payment transaction made against a student's fee record.
15	NOTIFICATION	A system-generated message sent to a user.
16	AUDIT_LOG	A security log recording all significant actions performed by users in the system.

2.2. DATA DICTIONARY

2.2.1. USER

Sr.	Name	Data Type	Constraint	Description
01	user_id (PK)	INT	NOT NULL, AUTO_INCREMENT	Unique identifier for each user
02	full_name	VARCHAR(100)	NOT NULL	Full name of the user
03	email	VARCHAR(100)	NOT NULL, UNIQUE	Login email address
04	password_hash	VARCHAR(255)	NOT NULL	Hashed password for authentication
05	role	ENUM	NOT NULL	Role: admin, teacher, or student
06	phone	VARCHAR(20)	NULL	Contact phone number
07	status	ENUM	DEFAULT 'active'	Account status: active or inactive
08	created_at	DATETIME	DEFAULT NOW()	Timestamp of account creation

2.2.2. STUDENT

Sr.	Name	Data Type	Constraint	Description
01	student_id (PK)	INT	NOT NULL, AUTO_INCREMENT	Unique identifier for each student
02	user_id	INT	NOT NULL, UNIQUE	Reference to USER table
03	dob	DATE	NULL	Date of birth
04	gender	ENUM	NULL	Gender: male, female, or other
05	program_id	INT	FK, NULL	Programme the student is enrolled in
06	enrollment_date	DATE	NULL	Date of initial enrolment
07	fee_status	ENUM	DEFAULT 'unpaid'	Current fee status: paid, unpaid, partial

2.2.3. SUBJECT

Sr.	Name	Data Type	Constraint	Description
01	subject_id (PK)	INT	NOT NULL, AUTO_INCREMENT	Unique identifier for the subject
02	program_id	INT	FK, NULL	Programme this subject belongs to

Sr.	Name	Data Type	Constraint	Description
03	teacher_id	INT	FK, NULL	Teacher assigned to this subject
04	subject_name	VARCHAR(100)	NOT NULL	Name of the subject
05	credit_hours	INT	NOT NULL	Credit hours assigned
06	fee	DECIMAL(10,2)	NOT NULL	Fee charged for this subject
07	prerequisite	VARCHAR(100)	NULL	Name of any prerequisite subject

2.3. BUSINESS RULES

- A user must have exactly one role: Admin, Teacher, or Student.
- A student can enroll in multiple subjects; a subject can have multiple students enrolled (M:N via enrollment table).
- A student belongs to exactly one programme at a time.
- A subject is taught by exactly one teacher, but a teacher can teach multiple subjects.
- A teacher can create multiple assignments and quizzes per subject.
- A student can submit at most one submission per assignment.
- A student can attempt a quiz, and the attempt is recorded with a score and timestamp.
- A live class generates attendance records for each enrolled student.
- A fee record is automatically generated when a student enrolls in a subject (via trigger).
- A fee record can have multiple partial payment transactions over time.
- Deleting a user cascades to their role-specific profile (admin, teacher, or student).
- Deleting a student cascades to their enrollments, fee records, attendance, and submissions.
- Deleting a programme sets the programme reference to NULL in associated students and subjects.
- All significant data operations are recorded in the audit_log for security compliance.
- Announcements are scoped to a specific subject or can be general (subject_id nullable).

2.4. ENTITY RELATIONSHIP DIAGRAM

The ERD for the AAMS database consists of **20 entities** organized into five functional modules: User Management, Academic Structure, Finance, Learning & Assessment, and Communication & Security.

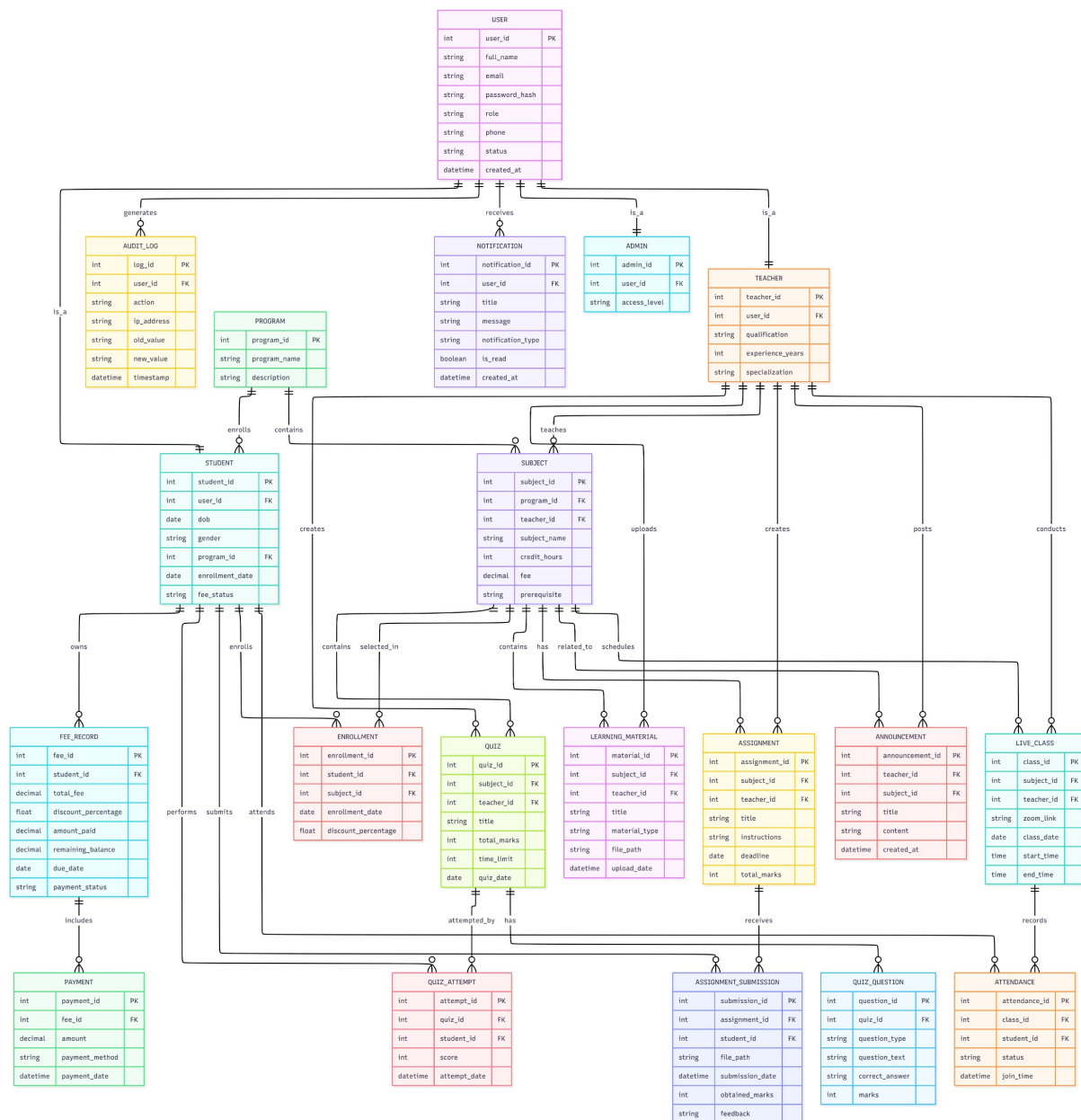


Figure 2.1: Entity Relationship Diagram — AAK Academy Management System

2.4.1. ERD Design Decisions

Design Decision	Rationale
Supertype/Subtype for users	Single USER table with role-specific tables (admin, teacher, student) avoids duplicate login credentials and simplifies authentication.
Separate enrollment table	Resolves the many-to-many relationship between students and subjects, allowing discount tracking per enrollment.
Separate payment table	One fee record can have multiple partial payments over time — normalizes the financial data.
Quiz questions as a separate entity	One quiz contains many questions; storing them in a child table supports normalization and flexible question management.
Attendance linked to live_class	Attendance is recorded per scheduled class session, not per subject directly, enabling accurate session-level tracking.
Audit log as standalone entity	Tracks all system changes independently for security compliance, decoupled from business tables.

CHAPTER 3: LOGICAL DATABASE DESIGN

3.1. RELATIONAL SCHEMA

3.1.1. Module 1: User Management

user(user_id, full_name, email, password_hash, role, phone, status, created_at)

- PK: user_id | UK: email

admin(admin_id, user_id, access_level)

- PK: admin_id | FK: user_id → user(user_id) ON DELETE CASCADE | UK: user_id

teacher(teacher_id, user_id, qualification, experience_years, specialization)

- PK: teacher_id | FK: user_id → user(user_id) ON DELETE CASCADE | UK: user_id

student(student_id, user_id, dob, gender, program_id, enrollment_date, fee_status)

- PK: student_id | FK: user_id → user(user_id) ON DELETE CASCADE
- FK: program_id → program(program_id) ON DELETE SET NULL | UK: user_id

3.1.2. Module 2: Academic Structure

program(program_id, program_name, description)

- PK: program_id

subject(subject_id, program_id, teacher_id, subject_name, credit_hours, fee, prerequisite)

- PK: subject_id | FK: program_id → program(program_id) ON DELETE SET NULL
- FK: teacher_id → teacher(teacher_id) ON DELETE SET NULL

enrollment(enrollment_id, student_id, subject_id, enrollment_date, discount_percentage)

- PK: enrollment_id | FK: student_id → student(student_id) ON DELETE CASCADE
- FK: subject_id → subject(subject_id) ON DELETE CASCADE

3.1.3. Module 3: Finance

fee_record(fee_id, student_id, total_fee, discount_percentage, amount_paid, remaining_balance, due_date, payment_status)

- PK: fee_id | FK: student_id → student(student_id) ON DELETE CASCADE

payment(payment_id, fee_id, amount, payment_method, payment_date)

- PK: payment_id | FK: fee_id → fee_record(fee_id) ON DELETE CASCADE

3.1.4. Module 4: Learning & Assessment

assignment(assignment_id, subject_id, teacher_id, title, instructions, deadline, total_marks)

assignment_submission(submission_id, assignment_id, student_id, file_path, submission_date, obtained_marks, feedback)

quiz(quiz_id, subject_id, teacher_id, title, total_marks, time_limit, quiz_date)

quiz_question(question_id, quiz_id, question_type, question_text, correct_answer, marks)

- FK: quiz_id → quiz(quiz_id) ON DELETE CASCADE

quiz_attempt(attempt_id, quiz_id, student_id, score, attempt_date)

learning_material(material_id, subject_id, teacher_id, title, material_type, file_path, upload_date)

live_class(class_id, subject_id, teacher_id, zoom_link, class_date, start_time, end_time)

attendance(attendance_id, class_id, student_id, status, join_time)

- FK: class_id → live_class(class_id) ON DELETE CASCADE
- FK: student_id → student(student_id) ON DELETE CASCADE

3.1.5. Module 5: Communication & Security

announcement(announcement_id, teacher_id, subject_id, title, content, created_at)

notification(notification_id, user_id, title, message, notification_type, is_read, created_at)

- FK: user_id → user(user_id) ON DELETE CASCADE

audit_log(log_id, user_id, action, ip_address, old_value, new_value, timestamp)

- FK: user_id → user(user_id) ON DELETE SET NULL

3.1.6. Relationship Cardinality Summary

Relationship	Type	Implementation
user → admin / teacher / student	1:0..1	FK in subtype table with UNIQUE constraint
program → student	1:N	program_id FK in student
program → subject	1:N	program_id FK in subject
teacher → subject	1:N	teacher_id FK in subject
student ↔ subject	M:N	enrollment junction table
student → fee_record	1:N	student_id FK in fee_record
fee_record → payment	1:N	fee_id FK in payment

Relationship	Type	Implementation
subject → assignment / quiz / material / live_class	1:N	subject_id FK
assignment → submission	1:N	assignment_id FK
quiz → question / attempt	1:N	quiz_id FK
live_class → attendance	1:N	class_id FK

3.2. FUNCTIONAL DEPENDENCIES

Key functional dependencies for the major tables are listed below using the notation $A \rightarrow B$.

USER:

$user_id \rightarrow full_name, email, password_hash, role, phone, status, created_at$
 $email \rightarrow user_id, full_name, role, status$

STUDENT:

$student_id \rightarrow user_id, dob, gender, program_id, enrollment_date, fee_status$
 $user_id \rightarrow student_id$

SUBJECT:

$subject_id \rightarrow program_id, teacher_id, subject_name, credit_hours, fee, prerequisite$

ENROLLMENT:

$enrollment_id \rightarrow student_id, subject_id, enrollment_date, discount_percentage$
 $(student_id, subject_id) \rightarrow enrollment_id$

FEE_RECORD:

$fee_id \rightarrow student_id, total_fee, discount_percentage, amount_paid, remaining_balance, due_date, payment_s$

ATTENDANCE:

$attendance_id \rightarrow class_id, student_id, status, join_time$
 $(class_id, student_id) \rightarrow status, join_time$

3.3. NORMALIZATION

All tables in the AAMS schema satisfy **Third Normal Form (3NF)** and most satisfy **Boyce-Codd Normal Form (BCNF)**.

3.3.1. First Normal Form (1NF)

Rule: All attributes must be atomic — no repeating groups or multivalued attributes.

Table	1NF	Example
user		Each user has one email, one role — no comma-separated values
enrollment		One row per student-subject pair (M:N decomposed into junction table)
quiz_question		Questions stored in separate rows, not as arrays within quiz
payment		Multiple payments stored as separate rows, not bundled in fee_record

Violation before 1NF (not used):

```
1 student_enrollment(student_id, student_name, subjects_enrolled)
2 -- subjects_enrolled = "DB Systems, OOP, Data Structures" [VIOLATION]
```

After 1NF:

```
1 enrollment(enrollment_id, student_id, subject_id, enrollment_date) [
  CORRECT]
```

3.3.2. Second Normal Form (2NF)

Rule: Must be in 1NF; no partial dependencies on a composite primary key.

Since all tables use surrogate integer primary keys (auto-incremented), partial dependencies are inherently eliminated. No non-key attribute depends on only part of the key.

Example — Partial dependency eliminated:

Student name depends only on `student_id`, not on the composite (`student_id`, `subject_id`) together. The name is stored in the `USER` table via `student.user_id`, not duplicated in the `enrollment` table.

3.3.3. Third Normal Form (3NF)

Rule: Must be in 2NF; no transitive dependencies (non-key attribute determines another non-key attribute).

Table	Transitive Dependency Removed	How
student	<code>student → program_name</code>	<code>program_name</code> stored in program table; only <code>program_id</code> FK in student
subject	<code>subject → teacher_name</code>	Teacher name in user table; only <code>teacher_id</code> FK in subject

Table	Transitive Dependency Removed	How
assignment	assignment $\rightarrow$ subject_name, teacher_name	Resolved via FK joins at query time
fee_record	fee $\rightarrow$ student_name	Student name retrieved via JOIN, not duplicated

Example — Transitive dependency removed:

```

1 -- BEFORE 3NF (violation):
2 subject(subject_id, subject_name, teacher_id, teacher_name,
   teacher_qualification)
3 -- teacher_name and teacher_qualification depend on teacher_id, not
   subject_id
4
5 -- AFTER 3NF (correct):
6 subject(subject_id, subject_name, teacher_id)
7 teacher(teacher_id, user_id, qualification, ...)
8 user(user_id, full_name, ...)

```

3.3.4. Boyce-Codd Normal Form (BCNF)

Rule: For every functional dependency $X \rightarrow Y$, X must be a superkey.

Table	BCNF	Notes
user		email $\rightarrow$ all attributes; email is a candidate key
program		program_id $\rightarrow$ all attributes
enrollment		enrollment_id is PK; (student_id, subject_id) is also a candidate key
fee_record		fee_id $\rightarrow$ all attributes; no non-trivial FDs on non-superkeys

3.3.5. Normalization Summary

Normal Form	Status	Key Action Taken
1NF	Satisfied	Decomposed M:N enrollments; separated quiz questions into child table
2NF	Satisfied	Used surrogate PKs; moved partial dependents to their parent tables
3NF	Satisfied	Removed all transitive dependencies via FK references
BCNF	Satisfied	All determinants are candidate or super keys in every table

CHAPTER 4: IMPLEMENTATION

4.1. LANGUAGE/Framework

The AAMS system is implemented using a **three-tier architecture**:

- **Frontend (Presentation Layer):** HTML5, CSS3, and vanilla JavaScript. A single-file application (`AAMS_RoleBased.html`) delivers the entire role-based GUI. Chosen for portability — no build tools required.
- **Backend (Application Layer):** Node.js with Express.js (`server.js`). Provides RESTful API endpoints consumed by the frontend. Selected for its non-blocking I/O and seamless JSON handling.
- **Database (Data Layer):** MySQL 8.x (`aak_academy` database). Chosen for its mature support of foreign key constraints, InnoDB transactions, stored procedures, and triggers.
- **Database Driver:** `mysql2` npm package with connection pooling (max 10 connections).
- **Configuration:** Environment variables managed via `dotenv` (`.env` file).

4.1.1. DBMS Configuration

Parameter	Value
DBMS	MySQL 8.x
Database Name	<code>aak_academy</code>
Database User	<code>aak_academy</code>
Host	<code>localhost</code>
Port	3306 (default)
Character Set	UTF-8
Storage Engine	InnoDB (supports FK constraints and transactions)

4.1.2. Constraints Implemented

Constraint Type	Count	Examples
PRIMARY KEY	20	All 20 tables
FOREIGN KEY	35+	Cascading deletes on dependent records
UNIQUE	4	<code>user.email</code> , <code>admin.user_id</code> , <code>teacher.user_id</code> , <code>student.user_id</code>
NOT NULL	30+	<code>full_name</code> , <code>email</code> , <code>role</code> , and other critical fields
ENUM	6	<code>role</code> , <code>status</code> , <code>gender</code> , <code>fee_status</code> , <code>payment_status</code> , <code>attendance.status</code>

Constraint Type	Count	Examples
DEFAULT	10+	created_at = CURRENT_TIMESTAMP, status = 'active'
ON DELETE CASCADE	20+	Deleting a user removes their admin/teacher/student profile
ON DELETE SET NULL	4	program_id, teacher_id when the parent record is deleted

4.1.3. Performance Indexes

```

1 CREATE INDEX idx_user_email ON user(email);
2 CREATE INDEX idx_student_program ON student(program_id);
3 CREATE INDEX idx_subject_program ON subject(program_id);
4 CREATE INDEX idx_subject_teacher ON subject(teacher_id);
5 CREATE INDEX idx_enrollment_student ON enrollment(student_id);
6 CREATE INDEX idx_enrollment_subject ON enrollment(subject_id);
7 CREATE INDEX idx_fee_student ON fee_record(student_id);
8 CREATE INDEX idx_payment_fee ON payment(fee_id);
9 CREATE INDEX idx_attendance_class ON attendance(class_id);
10 CREATE INDEX idx_attendance_student ON attendance(student_id);

```

Listing 4.1: Indexes for query optimization

4.2. DATABASE CONNECTIVITY

The frontend HTML file communicates with the MySQL database through an Express REST API running on Node.js. The connection is managed via a **mysql2 connection pool** with a maximum of 10 simultaneous connections.

4.2.1. Three-Tier Architecture Flow

```

1 const pool = mysql.createPool({
2   host:      process.env.DB_HOST,
3   user:      process.env.DB_USER,
4   password:  process.env.DB_PASSWORD,
5   database:  process.env.DB_NAME,
6   waitForConnections: true,
7   connectionLimit: 10,
8   dateStrings: true,
9 });

```

Listing 4.2: Connection pool configuration (server.js)

4.2.2. REST API Endpoints

Method	Endpoint	Description
GET	/api/health	Database connectivity check
POST	/api/login	User authentication (email + role)

Method	Endpoint	Description
GET	/api/data	Bulk load all data for dashboard
GET	/api/dashboard/stats	Admin dashboard live statistics
GET/POST/PUT/DELETE	/api/students	Student CRUD operations
GET/POST/PUT/DELETE	/api/teachers	Teacher CRUD operations
GET/POST/PUT/DELETE	/api/programs	Programme CRUD operations
GET/POST/PUT/DELETE	/api/subjects	Subject CRUD operations
GET/POST/PUT/DELETE	/api/enrollment	Enrollment CRUD operations
GET/POST/PUT/DELETE	/api/assignments	Assignment CRUD operations
GET/POST/PUT/DELETE	/api/quizzes	Quiz CRUD operations
GET/POST/PUT/DELETE	/api/liveclasses	Live class CRUD operations
GET/POST/PUT/DELETE	/api/attendance	Attendance CRUD operations
GET/POST/PUT/DELETE	/api/fees	Fee record CRUD operations
GET/POST/PUT/DELETE	/api/materials	Learning material CRUD
GET/POST/PUT/DELETE	/api/announcements	Announcement CRUD
GET/POST/PUT/DELETE	/api/users	User CRUD operations
GET	/api/notifications	Notifications list
GET	/api/auditlog	Audit log entries (read-only)

4.2.3. Security Measures

Measure	Implementation
Parameterized queries	All SQL uses ? placeholders — prevents SQL injection attacks
Role-based login	Email + role combination validated against database on every login
FK constraints	Prevents orphaned records from being created
Cascade deletes	Automatic cleanup of all dependent child records
Audit logging	All significant operations recorded in audit_log table

4.3. STORED PROCEDURES AND FUNCTIONS

#	Object Name	Type	Description	Input Parameters	Output
1	sp_CreateStudent	Stored Procedure	Creates a new user and student record atomically; logs the action in audit_log	p_name, p_email, p_phone, p_status, p_dob, p_gender, p_program_name, p_enrolled_date	None (INSERT)
2	sp_UpdateStudent	Stored Procedure	Updates an existing student's profile in both user and student tables; logs the update	p_student_id, p_name, p_status, p_dob, p_gender, p_program_name, p_enrolled_date	None (UPDATE)
3	sp_DeleteStudent	Stored Procedure	Deletes a student's user record, cascading to all child records; logs the deletion	p_student_id	None (DELETE)
4	trg_AutoGenerateFeeRecord	Trigger (AFTER INSERT on enrollment)	Automatically creates a fee_record when a student enrolls in a subject, computing the discounted balance	NEW.student_id, NEW.subject_id, NEW.discount_percentage	Inserts into fee_record

4.3.1. Stored Procedure: sp_CreateStudent

```

1 CREATE PROCEDURE sp_CreateStudent (
2     IN p_name VARCHAR(100),
3     IN p_email VARCHAR(100),
4     IN p_phone VARCHAR(20),
5     IN p_status VARCHAR(20),
6     IN p_dob DATE,
7     IN p_gender VARCHAR(20),
8     IN p_program_name VARCHAR(100),
9     IN p_enrolled_date DATE
10 )
11 BEGIN
12     DECLARE v_user_id INT;
13     DECLARE v_program_id INT;
14
15     SELECT program_id INTO v_program_id
16     FROM program WHERE LOWER(program_name) = LOWER(p_program_name) LIMIT
17     1;

```

```

18  INSERT INTO user (full_name, email, password_hash, role, phone,
19  status)
20  VALUES (p_name, LOWER(p_email), 'default_hash', 'student', p_phone,
21  LOWER(p_status));
22
23  SET v_user_id = LAST_INSERT_ID();
24
25  INSERT INTO student (user_id, dob, gender, program_id,
26  enrollment_date, fee_status)
27  VALUES (v_user_id, p_dob, LOWER(p_gender), v_program_id,
28  p_enrolled_date, 'unpaid');
29
30  INSERT INTO audit_log (user_id, action, old_value, new_value)
31  VALUES (1, 'INSERT student', NULL,
32  CONCAT('Student ', p_name, ' registered via sp_CreateStudent
33  '));
34  END;

```

Listing 4.3: Stored procedure for creating a student

4.3.2. Trigger: trg_AutoGenerateFeeRecord

```

1  CREATE TRIGGER trg_AutoGenerateFeeRecord
2  AFTER INSERT ON enrollment
3  FOR EACH ROW
4  BEGIN
5      DECLARE v_subject_fee DECIMAL(10,2);
6      DECLARE v_final_fee DECIMAL(10,2);
7
8      SELECT fee INTO v_subject_fee
9      FROM subject WHERE subject_id = NEW.subject_id;
10
11     SET v_final_fee = v_subject_fee
12     - (v_subject_fee * (NEW.discount_percentage / 100));
13
14     INSERT INTO fee_record
15     (student_id, total_fee, discount_percentage, amount_paid,
16     remaining_balance, due_date, payment_status)
17     VALUES
18     (NEW.student_id, v_subject_fee, NEW.discount_percentage, 0.00,
19     v_final_fee, DATE_ADD(CURDATE(), INTERVAL 30 DAY), 'unpaid');
20 END;

```

Listing 4.4: Trigger for automatic fee record generation on enrollment

4.3.3. Key SQL Queries

```

1  SELECT u.user_id, u.full_name, u.email, u.role, u.status,
2  a.admin_id, t.teacher_id, s.student_id
3  FROM user u
4  LEFT JOIN admin a ON u.user_id = a.user_id
5  LEFT JOIN teacher t ON u.user_id = t.user_id
6  LEFT JOIN student s ON u.user_id = s.user_id
7  WHERE u.email = ? AND u.role = ? AND u.status = 'active';

```

Listing 4.5: Login authentication query

```

1 SELECT s.student_id, u.full_name, p.program_name, s.dob, s.
   enrollment_date
2 FROM student s
3 JOIN user u ON s.user_id = u.user_id
4 LEFT JOIN program p ON s.program_id = p.program_id;

```

Listing 4.6: Student list with programme using JOIN

```

1 SELECT COALESCE(SUM(amount_paid), 0) AS total FROM fee_record;

```

Listing 4.7: Dashboard revenue aggregation

```

1 SELECT ROUND (
2     SUM(CASE WHEN status='present' THEN 1 ELSE 0 END) * 100.0 / COUNT(*)
3     , 1
4 ) AS attendance_rate
FROM attendance;

```

Listing 4.8: Attendance rate calculation

4.4. INTERFACES

4.4.1. GUI Design Philosophy

The GUI follows a **modern, clean, role-based dashboard** design with a consistent green/-gold colour scheme representing AAK Academy branding, fixed sidebar navigation, card-based dashboards, modal dialogs for CRUD forms, and toast notifications for user feedback.

4.4.2. Colour Palette

Variable	Hex	Usage
–green-deep	#1a4731	Sidebar, headers, primary buttons
–green-mid	#2d7a4f	Hover states and accents
–gold	#c9a227	Admin badges and highlights
–surface	#ffffff	Cards and modals
–danger	#c0392b	Delete buttons and errors
–success	#27ae60	Paid status tags and present attendance

4.4.3. Role-Based Navigation

Admin (17 pages): Dashboard, Students, Teachers, Programs, Subjects, Enrollment, Assignments, Quizzes, Live Classes, Attendance, Materials, Fees, Announcements, Notifications, Users, Audit Log.

Teacher (8 pages): Dashboard, My Subjects, Assignments, Quizzes, Live Classes, Mark Attendance, Materials, Announcements.

Student (9 pages): Dashboard, My Subjects, My Attendance, Assignments, Quizzes, Live Classes, Materials, My Fees, Announcements.

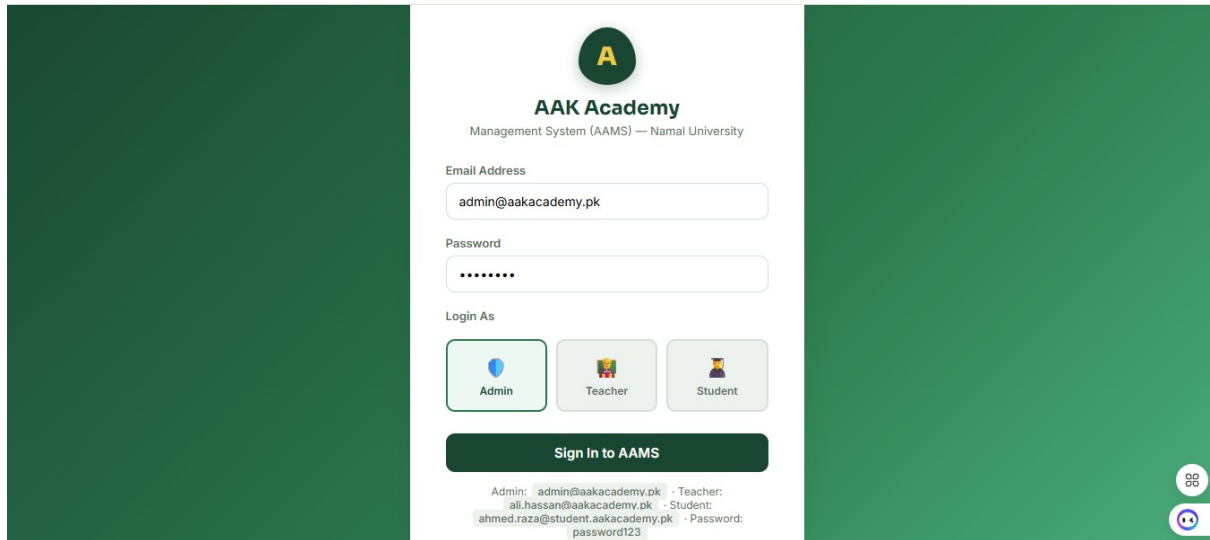
4.4.4. Sample Data Statistics

Table	Records
user	49
admin	2
teacher	23
student	24
program	24
subject	20
enrollment	28
fee_record	24
payment	21
assignment	22
assignment_submission	21
quiz	22
quiz_question	21
quiz_attempt	21
learning_material	21
announcement	22
live_class	22
attendance	22
notification	20
audit_log	20
Total	400+

4.4.5. Screenshots

Instructions: Run `npm start`, open `http://localhost:3000`, and capture the following system interfaces.

Login Screen



The login screen features a central white card on a dark green background. At the top of the card is the AAK Academy logo (a green circle with a yellow 'A') and the text 'AAK Academy Management System (AAMS) — Namal University'. Below this are input fields for 'Email Address' (containing 'admin@aakacademy.pk') and 'Password' (masked with dots). Under the password field are three 'Login As' buttons: 'Admin' (blue shield icon), 'Teacher' (teacher icon), and 'Student' (student icon). A dark green 'Sign In to AAMS' button is positioned below these. At the bottom, a list of login credentials is provided: Admin: admin@aakacademy.pk, Teacher: ali.hassan@aakacademy.pk, Student: ahmed.raza@student.aakacademy.pk, and Password: password123. A small chat icon is visible in the bottom right corner of the green background.

Figure 4.1: Main Login Screen

Admin Module

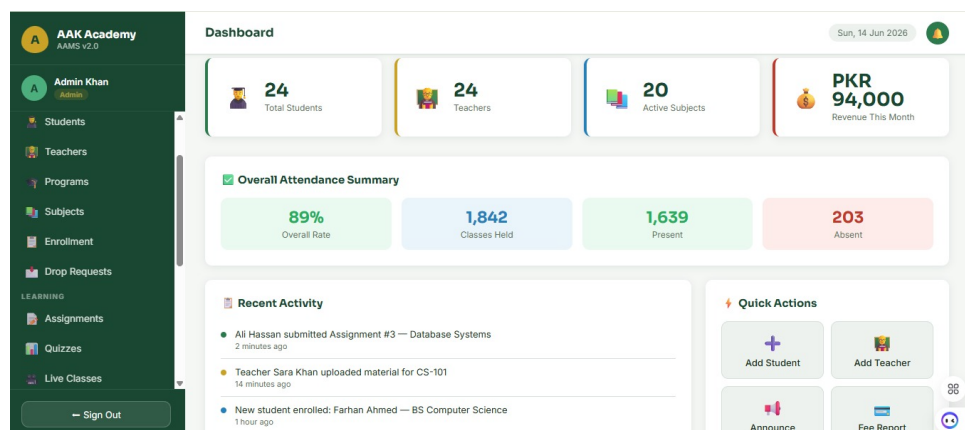


Figure 4.2: Admin Main Dashboard

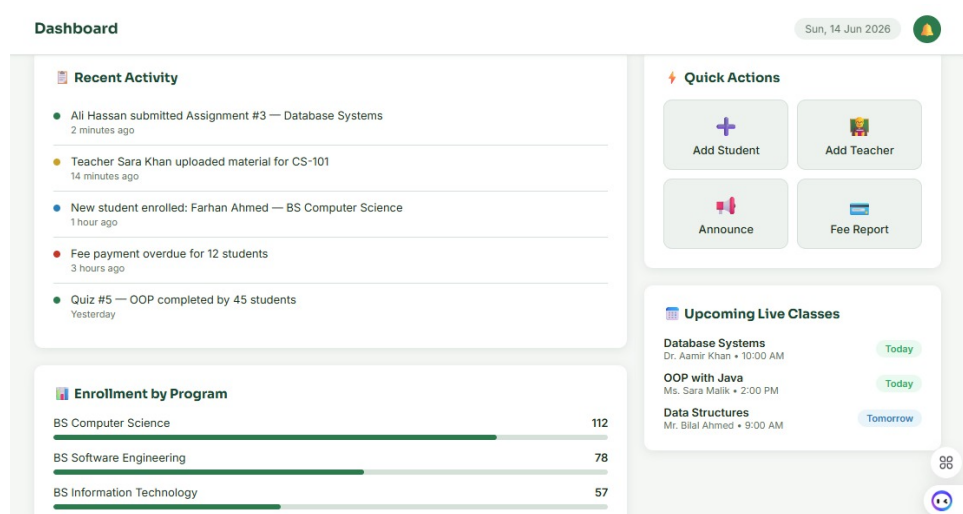


Figure 4.3: Admin Quick Actions and Recent Activity

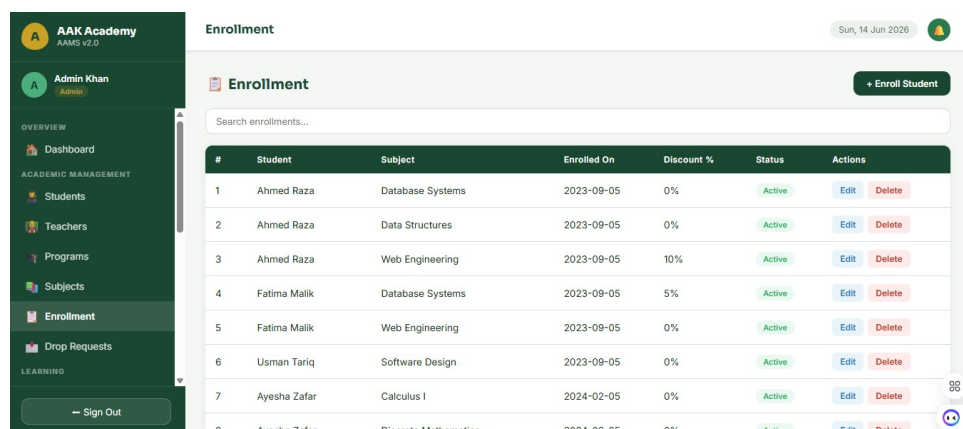
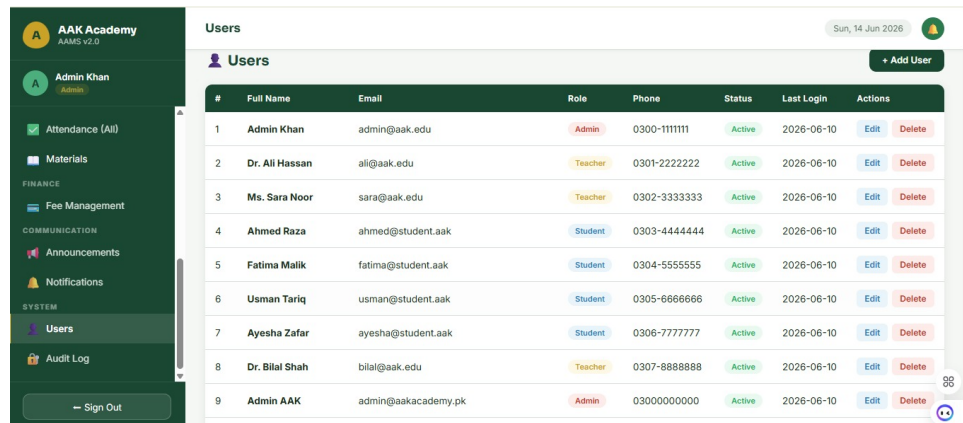


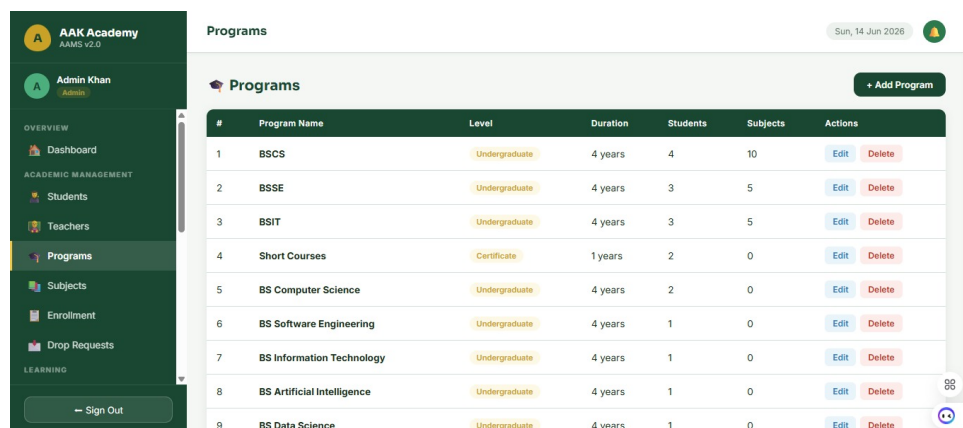
Figure 4.4: Student Enrollment (Admin Only)



Users

#	Full Name	Email	Role	Phone	Status	Last Login	Actions
1	Admin Khan	admin@aak.edu	Admin	0300-1111111	Active	2026-06-10	Edit Delete
2	Dr. Ali Hassan	ali@aak.edu	Teacher	0301-2222222	Active	2026-06-10	Edit Delete
3	Ms. Sara Noor	sara@aak.edu	Teacher	0302-3333333	Active	2026-06-10	Edit Delete
4	Ahmed Raza	ahmed@student.aak	Student	0303-4444444	Active	2026-06-10	Edit Delete
5	Fatima Malik	fatima@student.aak	Student	0304-5555555	Active	2026-06-10	Edit Delete
6	Usman Tariq	usman@student.aak	Student	0305-6666666	Active	2026-06-10	Edit Delete
7	Ayesha Zafar	ayesha@student.aak	Student	0306-7777777	Active	2026-06-10	Edit Delete
8	Dr. Bilal Shah	bilal@aak.edu	Teacher	0307-8888888	Active	2026-06-10	Edit Delete
9	Admin AAK	admin@aakacademy.pk	Admin	03000000000	Active	2026-06-10	Edit Delete

Figure 4.5: User Management (Admin Only)



Programs

#	Program Name	Level	Duration	Students	Subjects	Actions
1	BSCS	Undergraduate	4 years	4	10	Edit Delete
2	BSSE	Undergraduate	4 years	3	5	Edit Delete
3	BSIT	Undergraduate	4 years	3	5	Edit Delete
4	Short Courses	Certificate	1 years	2	0	Edit Delete
5	BS Computer Science	Undergraduate	4 years	2	0	Edit Delete
6	BS Software Engineering	Undergraduate	4 years	1	0	Edit Delete
7	BS Information Technology	Undergraduate	4 years	1	0	Edit Delete
8	BS Artificial Intelligence	Undergraduate	4 years	1	0	Edit Delete
9	BS Data Science	Undergraduate	4 years	1	0	Edit Delete

Figure 4.6: Program Management (Admin Only)

Fee Management

Sun, 14 Jun 2026

Fee Management + Generate Fee

PKR 4.8M Total Invoiced

PKR 3.2M Collected

PKR 1.6M Outstanding

#	Student	Total Fee	Discount	Amount Paid	Balance	Due Date	Status	Actions
1	Ahmed Raza	PKR 14,500	5%	PKR 14,500	PKR 0	2023-10-01	Paid	Edit Delete
2	Fatima Malik	PKR 9,500	0%	PKR 5,000	PKR 4,500	2023-10-01	Partial	Edit Delete
3	Usman Tariq	PKR 4,500	0%	PKR 4,500	PKR 0	2023-10-01	Paid	Edit Delete
4	Ayesha Zafar	PKR 8,000	0%	PKR 0	PKR 8,000	2024-03-01	Pending	Edit Delete
5	Muneeb Shah	PKR 5,000	0%	PKR 5,000	PKR 0	2024-03-01	Paid	Edit Delete
6	Anaya Siddiqui	PKR 4,800	5%	PKR 3,000	PKR 1,800	2024-03-01	Partial	Edit Delete
7	Rayan Iqbal	PKR 5,500	10%	PKR 5,500	PKR 0	2024-03-01	Paid	Edit Delete

Figure 4.7: Fee Management (Admin Only)

Audit Log

Sun, 14 Jun 2026

Audit Log

Search logs...

All Actions

#	User	Action	Old Value	New Value	Timestamp
1	Prof. Zain Ali	UPDATE quiz date	2024-03-13	2024-03-14	2024-03-13
2	Dr. Ayesha Khan	CREATE announcement	—	BI Presentation Notice	2024-03-12
3	Dr. Usman Raza	INSERT attendance	—	Attendance Marked	2024-03-11
4	Prof. Hina Tariq	UPDATE fee	Partial	Paid	2024-03-10
5	Prof. Bilal Ahmed	DELETE material	Old PDF	—	2024-03-09
6	Dr. Sara Noor	UPDATE quiz	20 Marks	25 Marks	2024-03-08
7	Dr. Ali Hassan	INSERT notification	—	Workshop Notification	2024-03-07
8	Admin AAK	UPDATE announcement	Draft	Published	2024-03-06
9	Dr. Bilal Shah	CREATE live class	—	Testing Live Class	2024-03-05

Figure 4.8: Audit Log (Admin Only)

The screenshot displays the 'Course Drop Requests' page in the AAK Academy Management System. The left sidebar shows the user is 'Admin Khan' (Admin) and lists navigation options: Overview (Dashboard), Academic Management (Students, Teachers, Programs, Subjects, Enrollment, Drop Requests), and Learning (Sign Out). The main content area is titled 'Course Drop Requests' and includes a warning message: 'Students cannot drop courses directly. They must email admin. Review requests below and take action.' Below this is a table with the following data:

#	Student	Subject	Reason	Requested On	Status	Actions
1	Salman Safar	Database Systems	Personal reasons	2026-06-10	Pending	Approve Reject
2	Zain Abbas	OOP	Workload too heavy	2026-06-12	Approved	View

Figure 4.9: Student Course Drop Requests (Admin Only)

The screenshot shows the 'Admin Main Functionalities' sidebar of the AAK Academy Management System. The sidebar is dark green and contains the following elements:

- AAK Academy AAMS v2.0
- User profile: Admin Khan (Admin)
- OVERVIEW: Dashboard
- ACADEMIC MANAGEMENT: Students (highlighted), Teachers, Programs, Subjects, Enrollment, Drop Requests
- LEARNING: Sign Out

Figure 4.10: Admin Main Functionalities

Teacher Module

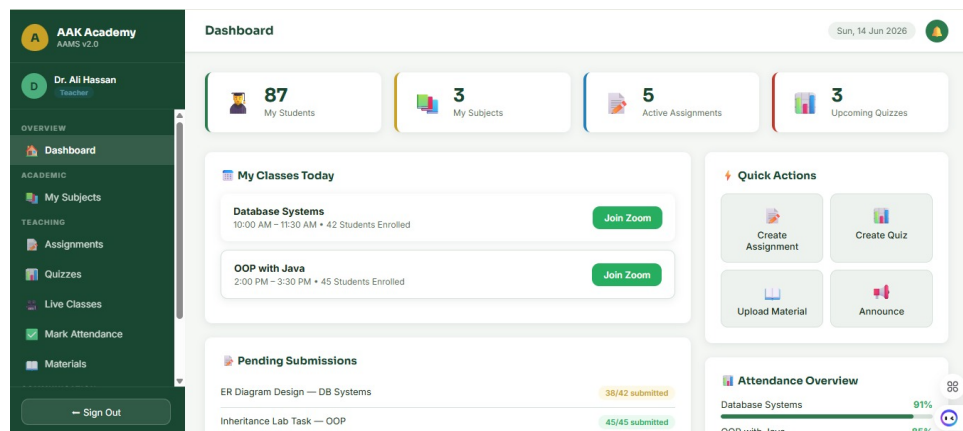


Figure 4.11: Teacher Main Dashboard

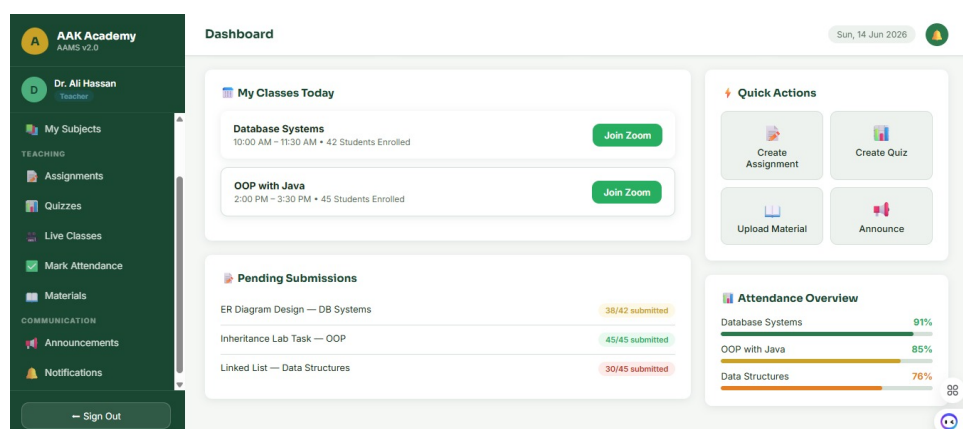


Figure 4.12: Teacher Quick Action Panel

My Subjects

Sun, 14 Jun 2026

My Subjects

Subjects are managed by Admin. Contact admin to add or remove a subject.

#	Code	Subject Name	Program	Credit Hrs	Students Enrolled
1	CS-271	Database Systems	BS Computer Science	3	42
2	CS-202	Object Oriented Programming	BS Computer Science	3	45
3	CS-211	Data Structures & Algorithms	BS Computer Science	3	45

Figure 4.13: Teacher Subjects

Assignments

Sun, 14 Jun 2026

Assignments

+ Create Assignment

My Assignments Submissions

#	Title	Subject	Deadline	Total Marks	Submissions	Actions
1	ER Diagram Design	Database Systems	2026-06-20	20	38	Edit Delete
2	Inheritance Lab Task	OOP	2026-06-18	15	45	Edit Delete

Figure 4.14: Teacher Assignment Section

Attendance

Sun, 14 Jun 2026

Attendance — My Classes + Mark Attendance

Search students... OOP

#	Student	Class	Date	Status	Actions
1	Ali Hassan	Database Systems	2026-06-14	Present	Edit
2	Salman Safar	Database Systems	2026-06-14	Present	Edit
3	Zain Abbas	OOP	2026-06-14	Absent	Edit

Figure 4.15: Teacher Attendance Marking

Live Classes

Sun, 14 Jun 2026

Live Classes + Schedule Class

#	Subject	Zoom Link	Date	Start	End	Actions
1	Database Systems	Join Zoom	2026-06-14	10:00 AM	11:30 AM	Edit Delete
2	OOP with Java	Join Zoom	2026-06-14	2:00 PM	3:30 PM	Edit Delete

Figure 4.16: Teacher Live Class Integration

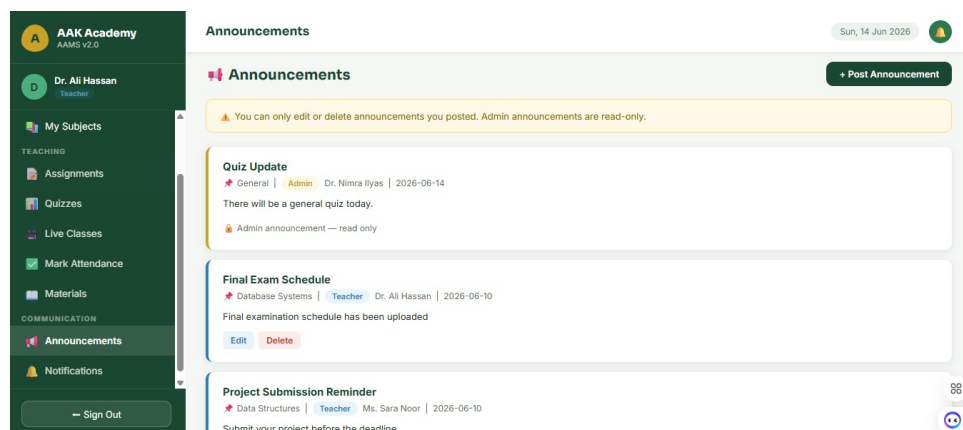


Figure 4.17: Teacher Announcements

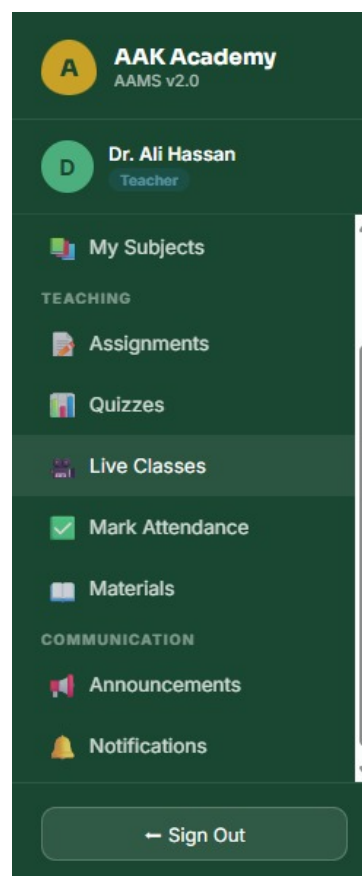


Figure 4.18: Teacher Main Functionalities

Student Module

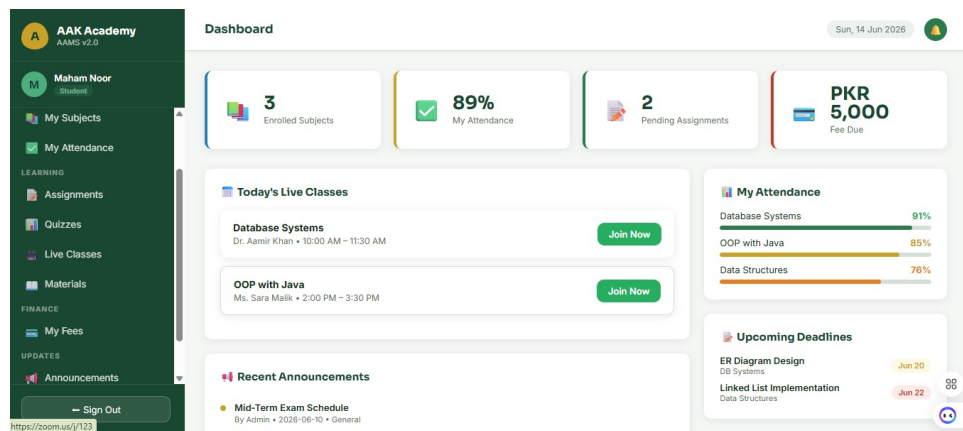


Figure 4.19: Student Main Dashboard

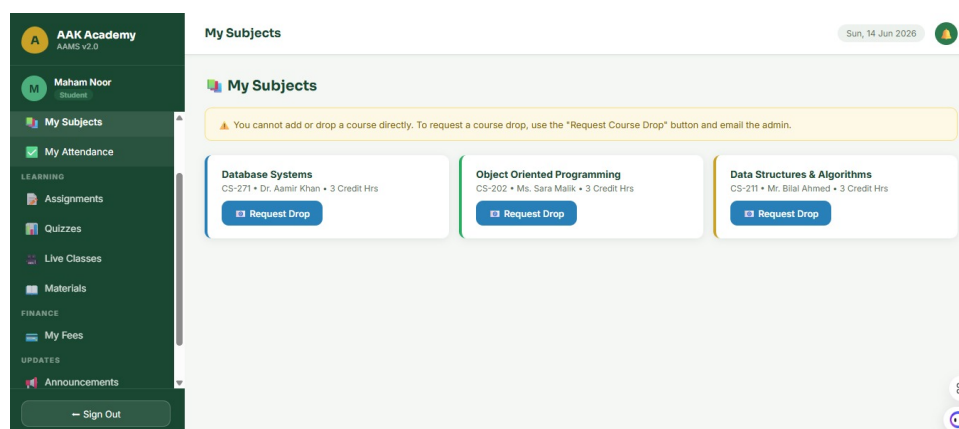


Figure 4.20: Student Subject Section

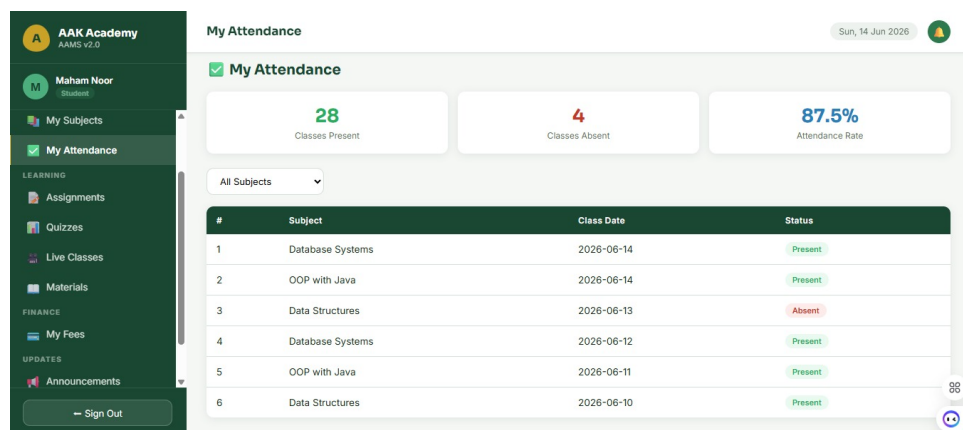


Figure 4.21: Student Attendance Section

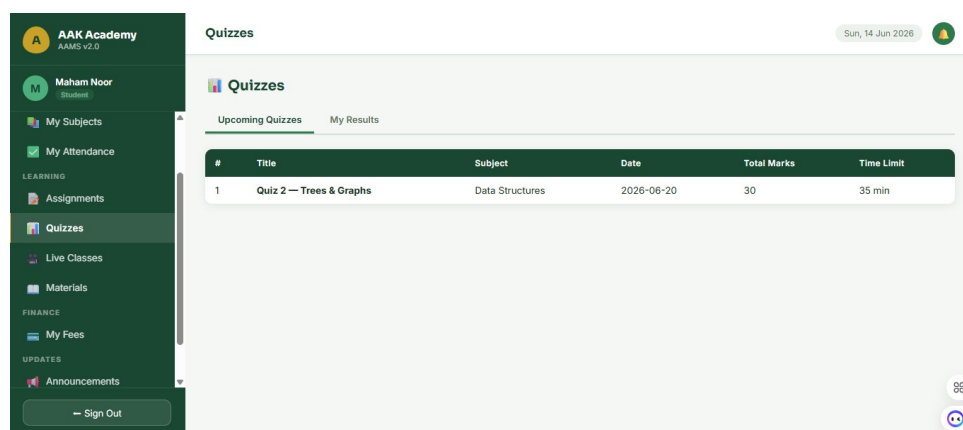


Figure 4.22: Student Quiz Section

My Fees

Sun, 14 Jun 2026

My Fee Status

PKR 15,000
Total Fee

PKR 10,000
Amount Paid

PKR 5,000
Balance Due

#	Month	Total Fee	Discount	Amount Paid	Balance	Due Date	Status
1	May 2026	PKR 15,000	10%	PKR 13,500	PKR 0	2026-05-25	Paid
2	June 2026	PKR 15,000	10%	PKR 8,500	PKR 5,000	2026-06-25	Partial

Figure 4.23: Student Fee Section

Notifications

Sun, 14 Jun 2026

Notifications

#	Title	Message	Date	Read
1	Assignment Graded	Your Assignment #1 — DB Systems has been graded	2026-06-11	No
2	Fee Reminder	Your June fee balance of PKR 5,000 is due by June 25	2026-06-10	Yes
3	Quiz Scheduled	Quiz 2 for Data Structures scheduled on June 20	2026-06-09	Yes

Figure 4.24: Student Notification Panel

The screenshot shows the 'Learning Materials' page for a student named Maham Noor. The left sidebar contains navigation links: My Subjects, My Attendance, Assignments, Quizzes, Live Classes, Materials, My Fees, and Announcements. The main content area displays a table of learning materials with columns for ID, Title, Subject, Teacher, Type, Upload Date, and Action. The table lists four items: a DB Lecture 1 (PDF), OOP Concepts Slides (PPT), a Linked List Video Tutorial (Video), and ERD Practice Problems (PDF). Each item has a corresponding action button (Download or Watch).

#	Title	Subject	Teacher	Type	Upload Date	Action
1	DB Lecture 1 — Intro to RDBMS	Database Systems	Dr. Aamir Khan	PDF	2026-05-15	Download
2	OOP Concepts Slides	OOP	Ms. Sara Malik	PPT	2026-05-18	Download
3	Linked List Video Tutorial	Data Structures	Mr. Bilal Ahmed	Video	2026-05-20	Watch
4	ERD Practice Problems	Database Systems	Dr. Aamir Khan	PDF	2026-06-01	Download

Figure 4.25: Student Learning Materials

The screenshot shows the 'Live Classes' page for the same student. The left sidebar is identical to the previous page. The main content area displays a list of scheduled live classes. Each class entry includes the subject, teacher, and time slot, along with a 'Join Zoom' button. The classes listed are: Database Systems (Dr. Aamir Khan, 10:00 AM - 11:30 AM, Sun, 14 Jun 2026), OOP with Java (Ms. Sara Malik, 2:00 PM - 3:30 PM, Sun, 14 Jun 2026), and Data Structures & Algorithms (Mr. Bilal Ahmed, 9:00 AM - 10:30 AM, Mon, 15 Jun 2026).

Subject	Teacher	Time Slot	Date	Action
Database Systems	Dr. Aamir Khan	10:00 AM - 11:30 AM	Sun, 14 Jun 2026	Join Zoom
OOP with Java	Ms. Sara Malik	2:00 PM - 3:30 PM	Sun, 14 Jun 2026	Join Zoom
Data Structures & Algorithms	Mr. Bilal Ahmed	9:00 AM - 10:30 AM	Mon, 15 Jun 2026	Join Zoom

Figure 4.26: Student Live Classes

TESTING & VALIDATION

DATABASE CONNECTIVITY TESTS

Test	Action	Expected Result	Status
Health check	GET /api/health	{“ok”: true}	Pass
Login — Admin	POST /api/login	Returns object with admin_id	Pass
Login — Teacher	POST /api/login	Returns object with teacher_id	Pass
Login — Student	POST /api/login	Returns object with student_id	Pass

CRUD OPERATIONS

Entity	Create	Read	Update	Delete
Programs				
Students				
Teachers				
Subjects				
Enrollment				
Fees				
Announcements				

REFERENTIAL INTEGRITY TESTS

Test Case	Action	Expected	Status
Delete user	DELETE user record	Cascades to admin/teacher/student	Pass
Delete student	DELETE student	Cascades enrollment, fee_record, attendance	Pass
Delete program	DELETE program	Sets program_id = NULL in student/subject	Pass
Invalid enrollment	Enroll without valid student_id	FK constraint error raised	Pass

CONCLUSION

The AAK Academy Management System successfully implements a **complete database-driven web application** that addresses the core challenges of manual academic management. The key achievements of this project are:

1. A **normalized 20-table relational schema** satisfying 3NF and BCNF across all entities.
2. **Full ERD-to-relational mapping** with proper FK constraints, cascade rules, and performance indexes.
3. A **role-based GUI** supporting Admin (17 pages), Teacher (8 pages), and Student (9 pages) workflows.
4. **Real-time database connectivity** via a Node.js/Express REST API with a connection pool.
5. **Complete CRUD operations** persisting immediately to MySQL with referential integrity.
6. **Automated database programmability** via stored procedures and triggers.
7. **Sample data** covering 400+ records across all 20 tables for all major academic and financial workflows.

The three-tier architecture (Presentation → Application → Data) ensures clean separation of concerns, maintainability, and scalability. The system can be extended in future with features such as file upload handling, email notifications, automated report generation, and enhanced stored procedures for complex business logic.

REFERENCES

- [1] R. Elmasri and S. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson Education, 2015.
- [2] C. J. Date, *An Introduction to Database Systems*, 8th ed. Addison-Wesley, 2003.
- [3] MySQL AB, *MySQL 8.0 Reference Manual*. Oracle Corporation, 2023. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- [4] Node.js Foundation, *Node.js Documentation*. [Online]. Available: <https://nodejs.org/en/docs/>
- [5] Express.js, *Express 4.x API Reference*. [Online]. Available: <https://expressjs.com/en/4x/api.html>
- [6] npm, *mysql2 package documentation*. [Online]. Available: <https://www.npmjs.com/package/mysql2>

APPENDIX: AI USAGE AND PROMPTS

This section documents the use of AI tools during the development of this project.

#	Prompt Used	Tool	Purpose
1	“Convert this ERD into a relational schema”	Google Gemini	Converting the ERD diagram into structured relational schema notation
2	[Add your prompts here]	[Tool name]	[Purpose]
3	[Add your prompts here]	[Tool name]	[Purpose]

Appendix A: How to Run the Project

```
1 # Step 1: Navigate to project folder
2 cd "d:\4th Semester\DB\aakc"
3
4 # Step 2: Install Node.js dependencies
5 npm install
6
7 # Step 3: Setup database (requires MySQL running)
8 node setup-db.js Namal.123
9
10 # Step 4: Start the application server
11 npm start
12
13 # Step 5: Open browser and navigate to:
14 # http://localhost:3000
```

Listing 4.9: Setup and run commands

Appendix B: Project Directory Structure

```
1 aakc/
2 |-- AAMS_RoleBased.html      # Frontend GUI (role-based)
3 |-- server.js                # Express REST API server
4 |-- setup-db.js              # Automated database setup script
5 |-- dbDDL.sql                # Schema (CREATE TABLE statements)
6 |-- dbDML.sql                # Sample data (INSERT statements)
7 |-- package.json             # Node.js configuration
8 |-- .env                     # Environment variables (DB credentials)
9 |-- .gitignore                # Git ignore rules
10 `-- screenshots/             # Screenshots folder
11     |-- 01_login.png
12     |-- 02_admin_dashboard.png
13     `-- ...
```

Listing 4.10: Project files and folders

Appendix C: Login Credentials (Test Accounts)

Role	Email	Password
Admin	admin@aakacademy.pk	password123
Teacher	ali.hassan@aakacademy.pk	password123
Student	ahmed.raza@student.aakacademy.pk	password123
Student	fatima.malik@student.aakacademy.pk	password123

Appendix D: Entity Count Summary

- **Total Entities (Tables):** 20
- **Total Foreign Key Relationships:** 35+
- **Total Performance Indexes:** 10
- **Total Sample Records:** 400+